

**MARRI LAXMAN REDDY INSTITUTE OF TECHNOLOGY AND
MANAGEMENT**

(AN AUTONOMOUS INSTITUTION)

Approved by AICTE, New Delhi & Affiliated to JNTUH, Hyderabad

Accredited by NAAC with 'A' Grade & Recognized Under Section 2(f) & 12(B) of the UGC act, 1956

Continuous Internal Examination - II (CIE-II), July-2026

Course: 2525804 - Advanced Computer Architecture (CSE)

H.T. No: 257X1D5805 Set No: 01 MLRS-MT25

Time: 02 Hours

Max. Marks: 30M

Note:

1. Question paper consists of Part-A and Part-B.
 2. In Part-A, answer all questions which carries 10 marks.
 3. In Part-B, answer any four questions which carries 20 marks.
-

PART - A (10 Marks)

Answer all questions. Each question carries 2 marks.

Q.No	Question	Marks	CO	BL
1 a)	Discuss the advantages of arithmetic pipelining over sequential arithmetic processing.	2M	CO3	BL2
1 b)	Explain the concept of cache coherence in multiprocessor systems.	2M	CO4	BL2
1 c)	Describe the key characteristics of the first-generation multi-computers.	2M	CO4	BL2
1 d)	Explain the concept of SIMD computer organization and its role in parallel computing.	2M	CO5	BL2
1 e)	Describe the major components of the CM-5 architecture.	2M	CO5	BL2

PART - B (20 Marks)

Answer any four questions. Each question carries 5 marks.

Q.No	Question	Marks	CO	BL
2	Illustrate how a non-linear pipeline processor executes an instruction sequence containing conditional branches.	5M	CO3	BL3
3	Analyze the performance of an instruction pipeline in the presence of structural, data, and control hazards.	5M	CO3	BL4
4	Analyze the performance of shared-memory and distributed-memory architectures for large-scale applications.	5M	CO4	BL4
5	Construct the architecture of a shared-memory multiprocessor and explain how processors communicate and synchronize.	5M	CO4	BL3
6	Compare vector processors and scalar processors in terms of architecture, execution model, and computational efficiency.	5M	CO5	BL4
7	Demonstrate the execution of a vector addition operation on a multivector multiprocessor.	5M	CO5	BL3

Solutions & Study Guide

PART - A Answers

Question 1 a) Discuss the advantages of arithmetic pipelining over sequential arithmetic processing.

Arithmetic pipelining divides complex arithmetic operations (like floating-point addition, subtraction, or multiplication) into multiple simple, cascaded physical stages separated by interface latches.

- **Increased Instruction Throughput:** Once the pipeline is full, the system outputs one complete arithmetic result per clock cycle, whereas sequential arithmetic processing requires the entire multi-cycle execution latency to finish before the next inputs can begin.
- **Optimal Hardware Utilization:** Instead of having a single monolithic execution unit active for a multi-cycle duration while other units are idle, all stages (such as exponent alignment, mantissa addition, normalization, and rounding) work on different datasets concurrently, maximizing functional unit utilization.

Question 1 b) Explain the concept of cache coherence in multiprocessor systems.

In shared-memory multiprocessor systems, each core is equipped with private high-speed caches (L1/L2) to reduce memory access latency.

- **The Coherence Problem:** If multiple processors cache copy(ies) of the same main memory address X , and one processor writes to its local cached copy, the copies in other processors' caches become stale (incoherent).
- **The Concept of Coherence:** Cache coherence is the set of protocols (like MESI or MOESI) and synchronization networks that guarantee any read operation to a shared memory address X returns the most recent write to X , either by invalidating (write-invalidate) or updating (write-update) stale copies in other caches.

Question 1 c) Describe the key characteristics of the first-generation multi-computers.

First-generation multicomputers (like the Caltech Cosmic Cube) are characterized by:

- **Distributed Memory & Message-Passing:** There is no single globally shared address space. Nodes consist of separate processing units with their own local physical memory. Inter-node communication is performed explicitly by sending and receiving messages.
- **Store-and-Forward Routing:** When a message is sent across the network, every intermediate node must receive and store the packet in its buffer completely before forwarding it to the next hop. This causes communication latency to be highly dependent on the physical distance (number of hops) between the source and destination nodes.

Question 1 d) Explain the concept of SIMD computer organization and its role in parallel computing.

- **SIMD Organization:** A Single Instruction Multiple Data architecture consists of a single central Control Unit (CU) and an array of Processing Elements (PEs). The CU fetches, decodes, and broadcasts a single instruction to all PEs, which execute it in lock-step on their own local data blocks.
- **Role in Parallel Computing:** SIMD is highly efficient for data-parallel applications (e.g., vector addition, matrix multiplications, and image processing). It eliminates the instruction fetch and decode overhead for individual elements and prevents control hazards (branches) since all processing elements perform the same instruction simultaneously.

Question 1 e) Describe the major components of the CM-5 architecture.

The Thinking Machines Connection Machine CM-5 supercomputer consists of two main components:

- **Processing Nodes:** Hundreds or thousands of nodes, each consisting of a SPARC processor, local memory, and custom vector arithmetic units to handle high-performance mathematical operations.
- **Dual Interconnection Networks:**
 - **Data Network:** Configured in a **Fat-Tree** topology, it provides high-bandwidth, low-latency point-to-point communication between nodes, thickening links toward the root to avoid communication bottlenecks.
 - **Control Network:** Handles global synchronization, broadcasting instructions, and reduction operations across processing nodes.

PART - B Answers

Question 2: Illustrate how a non-linear pipeline processor executes an instruction sequence containing conditional branches.

1. Non-Linear Pipeline Concept

Unlike linear pipelines where instructions progress sequentially through stages, a non-linear pipeline includes feedback loops and feed-forward bypass paths. This allows physical stages to be reused at different cycles for a single instruction. Reservation tables dictate stage reservations over time, and a **collision vector** manages scheduling to prevent multiple instructions from occupying the same stage at the same time.

2. The Impact of Conditional Branches

When a conditional branch is executed, the destination PC is not known until the instruction reaches the execution or write-back stages. This introduces a control hazard that interrupts the optimal scheduling sequence:

Cycle	1	2	3	4	5	6	7	8
Instr A (Branch)	IF	ID	EX	MEM	WB			
Instr B (Sequential)		IF	Stall	Flush				
Branch Target					IF	ID	EX	WB

3. Execution Sequence & Branch Penalty

1. **Instruction Fetch & Decode:** The branch instruction is fetched. The compiler tries to feed sequential instructions behind it.
2. **Evaluation & Stall:** If the branch outcome depends on an active calculation, the pipeline must wait.
3. **Taken Branch (Flush):** If the branch is predicted taken (or evaluated as taken), all instructions fetched sequentially after the branch (in the branch delay slots) must be flushed/invalidated.
4. **Target Rescheduling:** The pipeline is re-filled from the branch target address, incurring a **branch penalty** (wasted cycles / pipeline bubbles).

4. Optimization Strategies

To mitigate this performance degradation, non-linear pipelines implement:

- **Branch Prediction:** Hardware logic (like a Branch Target Buffer - BTB) predicts whether a branch is taken. If correct, execution continues without bubbles; if wrong, the pipeline is flushed.
- **Delayed Branching:** The compiler fills slots immediately following the branch with instructions that must be executed regardless of the branch outcome.

Question 3: Analyze the performance of an instruction pipeline in the presence of structural, data, and control hazards.

1. The Ideal Pipeline Case

In an ideal pipeline of k stages, processing N instructions takes:

$$T_{\text{ideal}} = [k + (N - 1)] \cdot \tau$$

where τ is the clock cycle time. The ideal CPI (Cycles Per Instruction) is 1.0.

2. Degradation by Pipeline Hazards

Hazards introduce stalls (bubbles), raising the CPI above 1.0. The actual execution time is modeled as:

$$T_{\text{actual}} = (N + k - 1 + \text{Stalls}) \cdot \tau$$

- **Structural Hazards (Resource Conflicts):**

- **Cause:** Occurs when hardware resources are shared. For example, if a pipeline has a single memory port, an instruction fetch (IF) stage and a memory data write (MEM) stage might try to access memory at the same time.
- **Impact:** The pipeline must stall the younger instruction for 1 cycle.
- **Analysis:** If memory operations constitute 30% of the instruction mix, and resource replication is absent, this hazard degrades the speedup factor by roughly 15 – 20%.

- **Data Hazards (Data Dependencies):**

- **Cause:** Read-After-Write (RAW), Write-After-Read (WAR), or Write-After-Write (WAW) dependencies. A RAW hazard occurs when Instruction B needs the output of Instruction A before A writes it back to the registers.
- **Impact:** Requires stalling the pipeline (data bubble) until the register is updated.
- **Mitigation Analysis:** Register forwarding/bypassing routes the ALU output directly back to the input latches, reducing the stall penalty from 2 cycles to 0 cycles for ALU-to-ALU dependencies.

- **Control Hazards (Branch Instructions):**

- **Cause:** Caused by branch or jump instructions where the target address is computed during the execution stage.
- **Impact:** Flushes the pipeline if the target prediction is incorrect, wasting 2 – 4 clock cycles per branch.
- **Analysis:** With a branch frequency of 20% and a branch penalty of 3 cycles, control hazards alone increase CPI by $0.2 \times 3 = 0.6$, dropping pipeline performance by 60%.

Question 4: Analyze the performance of shared-memory and distributed-memory architectures for large-scale applications.

1. Shared-Memory Architectures (SMP, NUMA)

- **Architecture:** Processors share a single global physical address space. They communicate implicitly by writing and reading shared memory variables.
- **Performance Analysis:**

- **Advantages:** Easy programming model (no explicit message-passing required; standard memory access works). Low latency for small-scale communication.
- **Scale Contention:** As the number of cores grows (> 128), memory access latency diverges (Non-Uniform Memory Access - NUMA factor).
- **Coherence Bottleneck:** The directory/bus cache-coherence traffic grows exponentially. The bandwidth limits of the system bus or switch become a performance ceiling.
- **Suitability:** Best for moderate-scale thread parallelism, database servers, and irregular memory access patterns.

2. Distributed-Memory Architectures (Message-Passing Multicomputers)

- **Architecture:** Each node is a self-contained computer with its own CPU and memory. Nodes communicate explicitly using packets over an interconnection network (e.g., via MPI).
- **Performance Analysis:**
 - **Scale Advantage:** Highly scalable. Systems can scale to thousands of nodes (e.g., supercomputers) because there is no global hardware cache-coherence overhead.
 - **Latency Constraints:** Inter-node communication involves software network stack overhead, making message passing much slower than local cache access.
 - **Programming Complexity:** Requires compilers or programmers to partition data and manage load balancing explicitly.
 - **Suitability:** Excellent for large-scale scientific simulations, grid computations, and embarrassingly parallel batch jobs where communication-to-computation ratios are low.

Question 5: Construct the architecture of a shared-memory multiprocessor and explain how processors communicate and synchronize.

1. Architecture of a Shared-Memory Multiprocessor

A symmetric shared-memory multiprocessor integrates multiple processors, each having private L1/L2 caches, linked via a coherent bus or crossbar interconnect to a shared main memory:

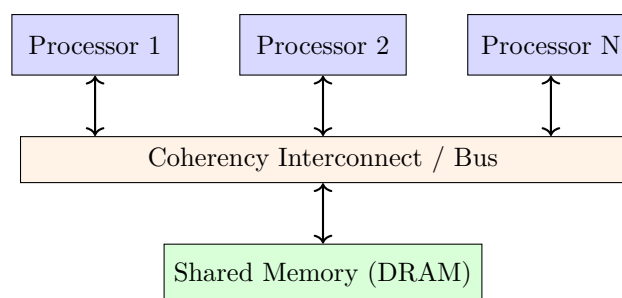


Figure 1: Symmetric multiprocessor architecture with shared memory and coherency interconnect.

2. Communication Mechanism

Communication is implicit. Processors read and write to the same shared address space. If Processor 1 writes a new value to a shared variable X at memory location $0x1000$, the cache

coherence protocol (e.g., MESI) changes the state of Processor 1's cache line to Modified (M) and sends an invalidate signal over the bus. When Processor 2 reads X , the coherence protocol intercepts the read, forces Processor 1 to write the modified data back, updates Processor 2's cache line to Shared (S), and serves the updated data, enabling communication.

3. Synchronization Mechanism

To prevent race conditions, hardware provides atomic synchronization primitives:

- **Atomic Instructions:** Operations like Compare-And-Swap (CAS) or Load-Link/Store-Conditional (LL/SC) execute read-modify-write sequences atomically.
- **Mutual Exclusion (Locks):** Processors acquire a lock (e.g., a spinlock) on a memory address. If the lock is held, the processor spins locally in its cache (reading the variable in Shared state). When the owner releases the lock (writing 0), the cache line is invalidated, prompting waiting processors to try to acquire the lock using atomic instructions.

Question 6: Compare vector processors and scalar processors in terms of architecture, execution model, and computational efficiency.

Vector and scalar processors differ fundamentally in how they handle data arrays:

Question 7: Demonstrate the execution of a vector addition operation on a multivector multiprocessor.

1. The Vector Addition Operation

Let us compute the vector sum:

$$\vec{C} = \vec{A} + \vec{B}$$

where the vectors contain $N = 256$ floating-point elements, and the hardware vector register length is $VL = 64$ elements.

2. Vector Loop Stripmining

Since the vector size ($N = 256$) exceeds the hardware register length ($VL = 64$), the compiler partitions the operation into 4 chunks (stripmining):

$$\text{Number of loops} = \frac{N}{VL} = \frac{256}{64} = 4 \text{ iterations}$$

Each iteration adds a subset of indices:

- Iteration 1: $i = 0 \dots 63$
- Iteration 2: $i = 64 \dots 127$
- Iteration 3: $i = 128 \dots 191$
- Iteration 4: $i = 192 \dots 255$

3. Execution on a Multiprocessor

In a **multivector multiprocessor**, these 4 loop iterations are distributed across different vector processors to execute in parallel:

Metric / Dimension	Scalar Processor	Vector Processor
Architectural Focus	Optimized for instruction-level parallelism (ILP). Features deep pipelines, branch predictors, and out-of-order execution logic.	Optimized for data-level parallelism (DLP). Features large vector register files (e.g., 64-256 elements per register) and pipelined vector ALUs.
Execution Model	SISD/MIMD: Fetches, decodes, and executes one instruction per scalar data element. Requires loop control code (increment, check, branch).	SIMD-like: A single vector instruction operates on an entire vector register (e.g., <code>VADD V1, V2, V3</code> performs 64 additions). Loop control overhead is eliminated.
Hazard Susceptibility	Highly vulnerable to data hazards (requires forwarding) and control hazards (requires branch prediction).	Data hazards are resolved once for the entire vector; elements are streamed sequentially through the pipelined ALU without stalls.
Memory Interface	Relies on deep cache hierarchies (L1, L2, L3) to hide memory latency.	Requires high-bandwidth memory pipelines, using techniques like memory interleaving, strided accesses, and gather-scatter operations.
Computational Efficiency	Highly efficient for irregular, branching, and general-purpose applications. Poor efficiency on massive numerical datasets.	Extremely efficient for regular, scientific computations (matrix arithmetic, signal processing). Inefficient for control-flow heavy code.

Table 1: Architectural comparison between scalar and vector processors.

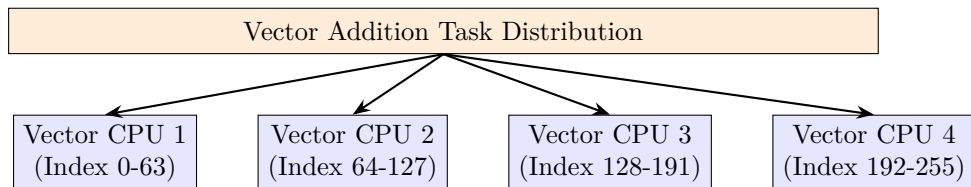


Figure 2: Parallel vector task distribution on a multivector multiprocessor.

4. Pipeline Execution Model within each Vector CPU

Within a single Vector CPU (e.g., Vector CPU 1 processing indices $0 \dots 63$):

1. **Vector Load:** Loads $V_A \leftarrow A[0 \dots 63]$ and $V_B \leftarrow B[0 \dots 63]$.
2. **Pipelined Execution:** The operation $VADD\ V_C, V_A, V_B$ is fetched. The elements stream through the vector adder pipeline:
 - **Cycle 1:** Element 0 enters stage 1 (exponent compare).
 - **Cycle 2:** Element 1 enters stage 1; Element 0 moves to stage 2 (mantissa add).
 - **Cycle 5 (steady state):** The pipeline is full (assuming 4 stages). Result element $C[0]$ is output, and subsequent results are output at a rate of **1 result element per cycle**.
3. **Vector Store:** The vector register V_C is written back to main memory $C[0 \dots 63]$.

By distributing the stripmined loops across the 4 vector processors, the entire addition is completed in parallel, achieving both spatial parallelism (across multiple processors) and temporal parallelism (via arithmetic pipelining).